



OPERATIONS TECHNOLOGY

Automated Test System 3.0

Requirements

Version: 1.03

Last Update: July 17, 2025

Author: Graham Harris



1 Contents

1 Contents2

2 List of Figures7

3 List of Tables8

4 Abstract9

5 Audience.....10

5.1 TERMINOLOGY 10

6 Background.....11

6.1 AUTOMATED TESTING CURRENT STATE (2025)..... 11

6.2 OPPORTUNITY..... 11

6.3 AUTOMATED TEST SYSTEM 3.0 12

 6.3.1 Decision Drivers12

7 High-Level Description.....13

7.1 GOALS..... 13

7.2 IN-SCOPE APPLICATIONS 14

7.3 IN-SCOPE METHODOLOGIES..... 14

 7.3.1 Regression Testing14

 7.3.2 Feature Testing15

 7.3.3 Unit Testing15

7.4 DELIVERY 15

7.5 FILE STORAGE..... 16

7.5.1	Approach	16
8	System Logical Design	17
8.1	OVERVIEW	17
8.2	TEST PACK	20
8.3	SYSTEM ENTITIES	21
8.3.1	Department	22
8.3.2	Application	22
8.3.3	Component	23
8.3.4	Feature.....	23
8.3.5	Workflow	24
8.3.6	Scenario	25
8.3.7	Action	27
8.3.8	Test Results	31
8.3.9	Test Case.....	31
8.4	FILE IMPORT EXAMPLE.....	33
8.5	INDEXING SYSTEM	33
9	User Interface.....	35
9.1	CONCEPTS.....	35
9.1.1	Active Application	35
9.1.2	System Heading	35
9.1.3	Colours.....	35
9.2	DATA DELETION	36

9.3	ROLES	36
9.4	SECTIONS	36
9.4.1	Application Manager	37
9.4.2	Environment Manager	37
9.4.3	Test Pack Manager	41
9.4.4	Test Run Manager	41
9.4.5	Test Case Manager	45
9.4.6	Testing Dashboard	47
10	Data Comparison	50
10.1	DATA COMPARE ACTIONS	50
10.1.1	File Compare Actions	51
10.2	DATA COMPARE VIEWER (DCV)	51
10.2.1	Debugging	52
10.2.2	Tolerances and Acceptable Differences	52
10.2.3	Break Reporting	53
10.2.4	Break List Viewer	56
10.3	COMPARE RESULTS TABLES	56
10.3.1	Beyond Compare	57
10.4	TEST RESULTS REPORT (TRR)	57
10.4.1	Summary Section	58
10.4.2	Tolerances and Acceptable Differences	59
10.4.3	Test Results	59

10.4.4	Log	59
11	Architecture	60
11.1	FILE / DOCUMENT STORAGE	60
11.2	EXCEL DOCUMENT FORMAT	61
12	System Usage	62
12.1	KEY CYCLES.....	62
12.1.1	Baseline Refresh Cycle	62
12.1.2	Expected Results Creation Cycle	62
12.1.3	Application Verification Cycle	63
12.2	SCENARIO DESIGN	63
12.2.1	Test Case Dependencies	63
12.2.2	Reliance on Baseline Data	63
12.3	FULL APPLICATION REGRESSION TESTS	64
13	Continuous Integration / Continuous Delivery (CI / DD).....	65
14	Test Case Migration	66
15	Project Plan	67
15.1	APPROACH	67
15.2	STRUCTURE	67
15.2.1	Phase 1 – Organisation User Interface	68
15.2.2	Phase 2 – Environment Manager	69
15.2.3	Phase 3 – Test Configuration User Interface	70
15.2.4	Phase 4 – Results Generator	72



15.2.5	Phase 5 – Test Runner.....	72
15.2.6	Phase 6 – Results Manager	73



2 List of Figures

Figure 1 Entity Relationship Diagram 21

3 List of Tables

Table 1	Definition of Terms	10
Table 2	Test Cases Hierarchy	19
Table 3	Test Pack Properties	20
Table 4	Department Properties	22
Table 5	Application Properties	22
Table 6	Component Properties	23
Table 7	Feature Properties	24
Table 8	Workflow Properties.....	25
Table 9	Scenario Properties	26
Table 10	Results Set Properties	31
Table 11	Test Case Properties	32
Table 12	User Interface Roles	36
Table 13	Environments Definition	38
Table 14	Environment Properties	39
Table 15	Environment Management Commands	40
Table 16	Break Report Definition	55
Table 17	Compare Results Table Definition	57
Table 18	Documents Table Definition	60

4 Abstract

This document provides the high-level scope and requirements for a new application called *Automated Test System 3.0*. Some architectural requirements are also described. Section 15 starting on page 67 provides a high-level project plan.

Automated Test System 3.0 (ATS3) is a replacement for Automated Test System 2.0 (ATS2) which in turn was a replacement for Automated Test System 1.0 (ATS1).

5 Audience

This document is written for technology staff within Operations Technology. Knowledge of AllianceBernstein, its applications, software development practises, tools, techniques, and testing processes is assumed.

5.1 Terminology

Since the Automated Testing System is itself an application, this document will distinguish between the ATS and applications it is used to test by referring to the ATS as a "*system*." Applications it is used to test are referred to as "*applications*."

Table 1 below provides a definition of some of the terms used throughout the document.

Term	Definition
Application	Business application the ATS is designed to verify.
Baseline Run	Execution of a collection of test cases where the version of the application software is the production version.
Run	Execution of a collection of test cases. A run maybe either a Baseline Run or a Test Run.
System	The Automated Test System or ATS.
Test Run	Execution of a collection of test cases where the version of the application software is the version to be verified.

Table 1 Definition of Terms

6 Background

6.1 Automated Testing Current State (2025)

Operations Technology uses an in-house system referred to as the *Automated Test System* or *ATS*. The purpose of the ATS is to facilitate regression testing of in-house applications.

There are currently two versions of the ATS in use. Version 1.0 (ATS1) is used for PMS. Version 2.0 (ATS2) is used for EPM, EPA1, EPE, and ESRM. Both versions are built around a third-party tool called Ranorex and use a combination of data stored in Excel and SQL Server and processing logic written in C# and shell scripts.

ATS2 is an improvement over ATS1 because it has a less complex solution for implementing test case logic.

6.2 Opportunity

ATS2 presents a series of opportunities:

- It is not intuitive to use due to the complexity of the "*user interface*." Users require sustained, detailed training to be able to manage regression testing.
- Adding new test cases is time-consuming and requires special training.
- The "*user interface*" is not implemented in a single place. Some features have been added to a web-based interface built using ASP.NET. Much of the rest of the "*user interface*" involves maintaining a complex set of Excel spreadsheets and a complex network of dependencies between documents, tabs, and values in individual cells.
- The complex architecture makes the system time consuming and challenging to extend. It makes it time-consuming to apply to new business applications.
- The benefits of having Ranorex at the center of the system do not outweigh the costs in terms of additional complexity. The regression testing requirements of Operations Technology applications go beyond Ranorex's limits.

- The architecture does not currently allow ATS2 to easily be integrated with DevOps or GitHub Actions.
- It does not seem possible to implement "*push button*" environment refreshes and resets, which has been a goal since the inception of ATS2.
- Investigation of breaks is very time consuming. Often the first task is to determine whether the test case broke due to an issue within the application under test or whether the break has been caused by reliability issues within ATS2 itself.
- Comparison of data compare results is time consuming.

6.3 Automated Test System 3.0

Following a series of deep-dive meetings during June 2025 a decision was made to replace ATS2 with a new solution called ATS3.

ATS2 will move into "*keep-the-lights-on mode plus*". This means ATS2 will continue to be used for application regression testing, and new test cases may be added as necessary. The completely new system, ATS3, will be designed and built while ATS2 is in use as the primary. Work on enhancing ATS2 is to cease. Defects which prevent applications being tested can be fixed.

When ATS3 is sufficiently mature test cases will be migrated from ATS2 to ATS3. Section 14 discusses the approach to test case migration and associated challenges. Once test cases have been migrated applications will switch to using ATS3 instead of ATS2.

The final goal is to decommission ATS2. ATS1 will continue to be used for PMS until PMS end-of-life.

6.3.1 Decision Drivers

The fundamental design of ATS2 makes use of the system overly complex and time-consuming and this cannot be resolved without a complete redesign and rewrite. Ranorex does not provide sufficient value-add, and the other challenges described in the section 6.2 will either be too expensive to resolve or may not be resolvable using the current design.

7 High-Level Description

ATS3 is to be a completely new system. It will:

- Have its own, new source code repository and will not share anything with ATS1 or ATS2. This is so it will be easy to decommission ATS1 and ATS2.
- Be written in C# .NET using Visual Studio.
- Have its own, new SQL Server storage tier. This must be distinct and not shared with ATS1 and ATS2.
- Have a web-based ASP.NET user interface.
- Use log4net to write logging messages to SQL Server.

7.1 Goals

These are the key goals of the new system:

- Reduce the time required to:
 - Create and update test cases.
 - Manage environments.
 - Analyse test results and locate root causes of breaks.
- Reduce the time to complete a full regression test of EPA. As of June 2025, a full cycle takes around two weeks to complete. This time comprises around two days to run the test and eight days to analyse the results.
 - First Milestone – Reduce the time required for a full cycle to one week.
 - Second Milestone – Reduce the time required for a full cycle to three days.
- The user interface must be easy to understand and self-explanatory. This is to keep the amount of training required to use the system to an absolute minimum.

- It should not be necessary to consult separate documentation or a manual to use ATS3. If documentation is necessary, it should be embedded within the user interface, for example by making use of tooltips, clear labelling, and dialogue boxes.

7.2 In-Scope Applications

The following applications will use ATS3:

- Enterprise Performance Engine (EPE)
- Enterprise Portfolio Accounting 1 (EPA1)
- Enterprise Portfolio Accounting 2 (EPA2)
- Enterprise Price Master (EPM)
- Enterprise Security Reference Master (ESRM)

7.3 In-Scope Methodologies

This section describes the types of test methodology ATS3 will be able to support. User Acceptance Testing and Integration Testing are out-of-scope.

7.3.1 Regression Testing

ATS3 will primarily be used for regression testing.

ATS2 considers two classes of regression test referred to as *Scenario Testing* and *Simulation Testing*. Scenario tests seek to verify individual application features. Simulation tests focus on overnight data processing cycles which are run for multiple consecutive days. Simulation tests can be run for one or more days and are often run for more than one month's worth of processing.

7.3.1.1 ATS3 Nomenclature

Simulation Testing can be considered as a type of *Scenario Test*. To simplify system design and usage, Simulation Tests will become a class of Scenario Test in ATS3. See section 8

starting on page 17 for further information. *Simulation Tests* will no longer be dealt with as separate and parallel to *Scenario Tests*.

7.3.2 Feature Testing

ATS2 may also be used for feature testing but its use is not mandatory. Where it is more efficient to carry out feature testing manually this is permitted.

When a release version of an application is being regression tested prior to deployment to production, regression test cases will need to be updated to match changes in application behaviour resulting from feature additions and changes. The changes to existing regression test cases, and new test cases added will in any case constitute feature tests. Once the release has been deployed to production, the updated test cases become part of the regression test pack. A "test pack" is the set of test scenarios which work for a specific version of an application.

7.3.3 Unit Testing

ATS3 will not be used for unit testing. All in-scope applications (see section 7.1) will implement unit testing using xUnit or a similar unit test framework.

7.4 Delivery

The goals of ATS3 relate to simplification of the processes for creation and management of test cases and improvement in productivity surrounding test execution and results analysis. These factors are all dependent on the quality of the user interface.

The focus for ATS1 and ATS2 has been to build a test framework around Ranorex. By focusing less on usability, the existing systems do not have coherent UIs, and users must step between different views and technologies which impacts productivity.

The focus for ATS3 will be the user interface. The UI will be built first, using an Agile inspired methodology. The rest of the system will hang from the UI.

7.5 File Storage

ATS1 and ATS2 rely heavily on input files and Excel documents stored in the file system. QA team members manage the system by navigating to various folders (which they must know the location of) to edit and use files. This arrangement has an adverse effect on productivity, system usability, and reliability.

7.5.1 Approach

ATS3 will not use the file system for file and document storage. Instead, input files and documents will be stored in the ATS database. The UI will be designed to make it easy to upload, view, edit, and delete documents. Access to files and documents will be embedded in-situ within screens where the files are relevant. This will improve team productivity.

For example, where a test case requires an Excel document to supply input values to a test scenario, the screen used to edit the test case will allow the document to be uploaded, replaced, viewed, edited, and deleted.

Section 11.1 starting on page 60 provides the high-level architecture for the file storage system. Section 11.2 describes the standard format all Excel documents will be constrained to follow.

8 System Logical Design

This section describes the logical pieces of the system and how they fit together. From this point on, logical parts of the system will be written using capital letters (EG "Test Case").

8.1 Overview

Departments, Applications, Components, and Features can be created and configured by users. Their purpose is to allow test Scenarios to be organised into groups.

Scenarios are created by users and configured to have collections of Workflows. Workflows are collections of Actions. Scenarios also have a configurable set of Test Cases.

Scenarios can be executed at the Application, Component, Feature, and Scenario level. Individual Test Cases within Scenarios can be executed in isolation.

Once a Workflow has been configured by assigning a list of Actions, a Scenario can be defined which uses the Workflow. Scenarios supply global Input Values to Actions within the Workflow. Test Cases supply specific Input Values to Actions.

Workflows are configured by creating sequenced lists of Actions. Examples of Workflow are:

- Import File
- Table to Table Data Copy
- Process Data
- Export File

Workflows have a flag indicating whether they can be processed in parallel. Some Action types also support parallel processing.

Departments, Applications, Components, Features, Scenarios, and Test Cases are all "*soft*" and are created, edited, configured, and deleted via the UI.

Actions are "*hard*" programmed into the ATS. They are built by developers and testers using C# .NET in Visual Studio. Once different types of Actions have been programmed, they can be assembled into Workflows through the UI.

Test Cases supply Input Values to Actions. They are created and configured within the UI. The content of Test Cases is defined by the Actions in the Workflow the Test Case relates to. When editing a Test Case, the set of fields displayed in the UI is determined dynamically based on the Actions within the Workflows of the Scenario.

Expected Results are generated by running Scenarios against an Environment where the production version of the Application is installed. Actual Results are generated by running Scenarios against an Environment where a version of the Application to be verified is installed.

Test Packs provide versioning for Scenarios, Test Cases, and Expected Results.

Scenarios and Test Cases are organised in a hierarchy. This makes the application easier to understand and makes it easier to extend use of ATS3 to more applications. Table 2 shows levels within the hierarchy. It is important the same nomenclature is used within the ATS source code and user interface.

Level	Name	Examples
1	Department	• Operations Technology • Fixed Income Technology • Equities Investment Management Technology
2	Application	• Enterprise Performance Engine (EPE) • Enterprise Portfolio Accounting 1 (EPA1) • Enterprise Portfolio Accounting 2 (EPA2) • Enterprise Price Master (EPM) • Enterprise Security Reference Master (ESRM)
3	Component	• User Interface • Processing • Storage • Importers

Level	Name	Examples
		- Exporters - Reports
4	Feature	- Transaction Entry Screens - Portfolio Builder - Security Matcher - Corporate Actions Processor
5	Scenario	- Edit Transaction (UI) - Rebuild Account (Processing) - Insert New Position (Storage) - Import Prices (Importers) - Export Lots (Exporters) - Run Valuation Report (Reports)
6	Action	- Execute SQL Statement - Execute Shell Command - Compare Data
7	Test Case	For Component "<i>Processing</i>" and Scenario "<i>Rebuild Account</i>": - Rebuild Account abc123 for today - Rebuild Account abc123 for yesterday - Rebuild Account abc123 for the last 3 days - Rebuild Account xyz456 for the last month

Table 2 Test Cases Hierarchy

8.2 Test Pack

The collection of Scenarios and Test Cases used to test one version of an application are likely to report false breaks if used to test a different version of the application. When the version number of an application is changed it is because changes to the application's code have been made to fix defects and implement new behaviour and changes to behaviour. The changes in actual results will not match the expected results incorporated into the Scenarios and Test Cases. This means the Scenarios, Test Cases, and Expect Results must all be updated to match changes to the application.

A Test Pack is the collection of Scenarios, Test Cases, and Expected Results which relate to a given version of an application. When a new version of the application is to be tested, the Test Pack for the previous version of the application must be copied and modified.

Test Packs for older versions of the application can be archived.

Name	Description
Application	Reference to the application the Test Pack belongs to.
Code Branch Name	Name that matches a code branch in the source code repository.
Version	Name of the Fix Version for the code branch.
Code Branch URL	Link to source code.
Version URL	Link to Fix Version in Jira.

Table 3 Test Pack Properties

8.3 System Entities

This section describes the logical entities the system is comprised on.

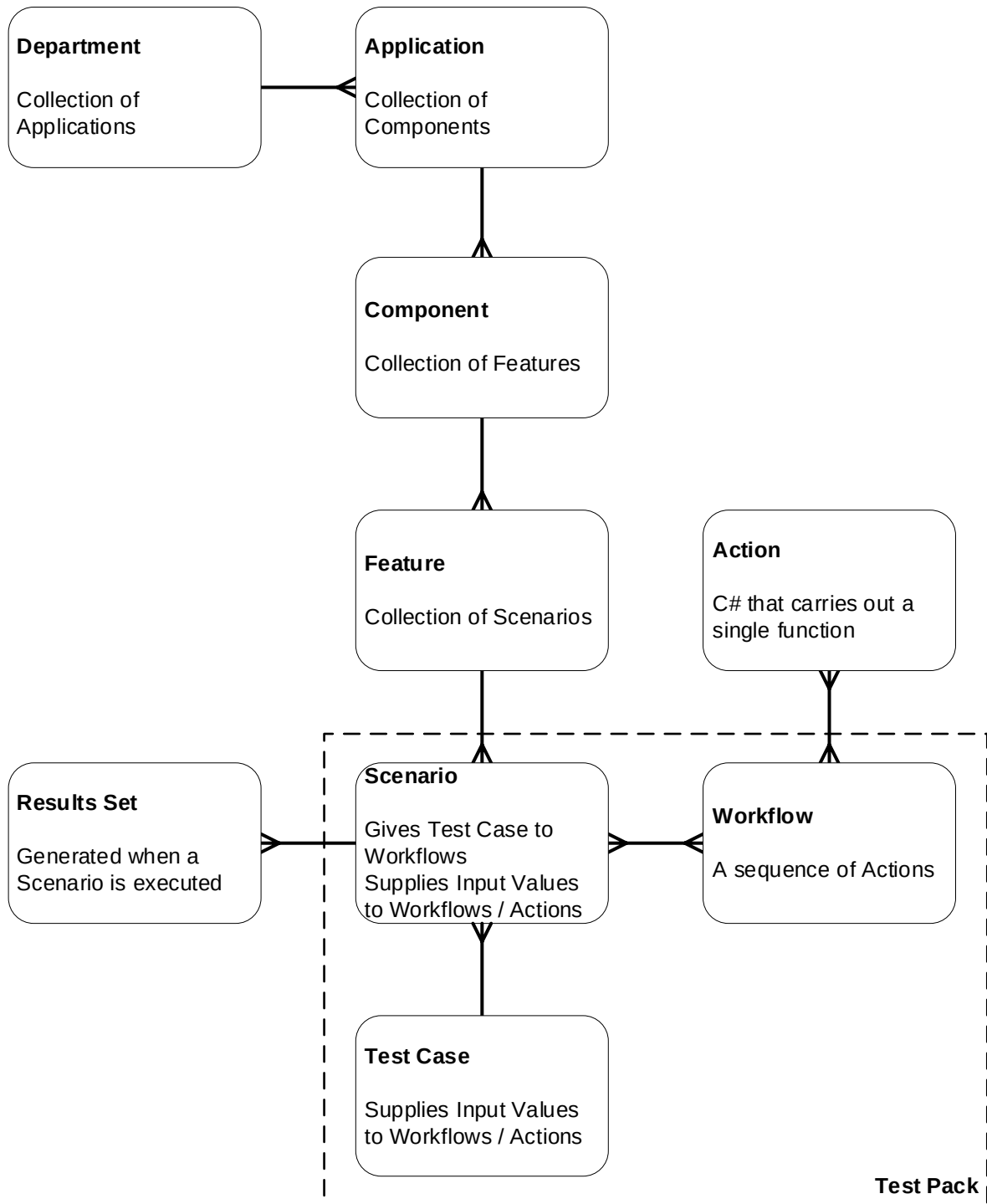


Figure 1 Entity Relationship Diagram

8.3.1 Department

The user interface allows Departments to be created, edited, and removed. Departments can only be removed if there are no applications associated with them.

NameDescription	
Name	A short name of the department.
Description	Description of the department.
Manager	Name of the AB IT manager of the department.

Table 4 Department Properties

8.3.2 Application

The user interface allows Applications to be created, edited, and removed. Removing an Application causes everything associated with the Application to be removed (Scenarios, Test Cases, Test Results, etc.)

NameDescription	
Name	Short name of the application.
Acronym	The acronym used to refer to the application (EPE, EPA, EPM, Etc.)
Description	Description of the application.
Manager	Name of the AB IT manager of the application.
Department	Reference to the Department the Application is a member of.

Table 5 Application Properties

8.3.3 Component

Applications are sub-divided into Components. Components are collections of Features. The user interface allows Components to be added to, edited, and removed from Applications.

When a Component is removed from an Application all the Scenarios, Test Cases, Test Results, etc. associated with the Component are deleted.

Name Description	
Name	Short name of the component.
Description	Description of the component.
Acronym	Values such as UI, DB, and APP will be used. These values are required by the Indexing System (section 8.5).
Manager	Name of the IT manager of the Component (AB or off-shore).
Application	Reference to the Application the Component is a member of.

Table 6 Component Properties

8.3.4 Feature

Components are sub-divided into Features. Features are collections of test Scenarios. The user interface allows Features to be added to, edited, and removed from Components.

When a Feature is removed from a Component all the Scenarios, Test Cases, Test Results, etc. associated with the Feature are deleted.

Name Description	
Name	Short name of the Feature.
Description	Description of the Feature.
Acronym	Values such as TXN, PBD, and CA. These values are required by the Indexing System (section 8.5).
Manager	Name of the IT manager of the Feature (AB or off-shore).
Component	Reference to the Component the Feature is a member of.

Table 7 Feature Properties

8.3.5 Workflow

Workflows are isolated by Application. The Workflows for one Application are separate from those of other Applications.

Workflows are created and edited in the user interface. Users assemble them by selecting Actions and placing the Actions in a sequence. When Actions are added to a Workflow users may check or uncheck the "*Run Parallel*" checkbox for any Action that supports parallel processing. This checkbox is greyed out for Actions that do not support parallel processing.

Workflows themselves have a "*Run Parallel*" property. If this value is set to true, the Workflow will run in parallel to subsequent Workflows listed within the same Scenario.

When a Scenario runs its Workflows are executed in sequence. Any Workflow that has parallel processing switched on does not block the next Workflow in the sequence from starting. The next Workflow will be started before the parallel processing Workflow has completed.

When Workflows are executed the sequence of Actions they are configured to run is repeated for each Test Case within the Scenario. The Scenario and the Test Cases supply the Input Values to the Actions.

Name	Description
ID	An automatically generate ID number.
Name	Short name of the Workflow.
Description	Description of the Workflow.
Creation Details	User, date, time.
Last Update	User, date, time.
Actions	A sequenced list.
Application	Reference to the Application the Workflow belongs to.
Run Parallel	Allows the Workflow to run in parallel to subsequent Workflows within a Scenario.

Table 8 Workflow Properties

8.3.6 Scenario

Scenarios form the intersection between Workflows and Test Cases. Workflows define the Actions that will be run, and the Scenario provides a list of Test Cases which will supply Input Values to the Actions. When a Scenario is executed, the Workflows are repeated once per Test Case.

Scenarios have Scenario Properties. Scenario Properties supply Input Values to Actions within Workflows within the Scenario. Scenario Properties make it possible to provide identical Input Values to Actions across all Test Cases within the Scenario.

Name	Description
ID	An automatically generate ID number that is unique within the Application the Scenario belongs to.
Name	Short name of the Scenario.
Description	Description of the Scenario.
Creation Details	User, date, time.
Last Update	User, date, time.
Workflows	List of references to the sequence of Workflows the Scenario uses. Each reference is assigned a category (Setup, Run Test, Compare, Teardown). Workflows are grouped within the Scenario by category.
Test Cases	Sequenced list of Test Cases that supply Input Values to the Actions within the Workflows of the Scenario.
Scenario Properties	A collection of properties visible to all Actions within all Workflows associated with the Scenario.
Test Pack	Reference to the Test Pack the Scenario belongs to.
Comment History	A collection of comments added by users. Each comment has a creation date / time / user stamp.
Tracking Keys	A collection of key / URL pairs. The key may contain Jira, ServiceNow, etc. keys and the URL is a link to the ticket in the associated tracking platform.
Run Parallel	Boolean flag that causes the Test Cases in the Scenario to be run in parallel.
Maximum Processes	Maximum number of Test Cases to run in parallel at once. If "Run Parallel" is false, this value is forced to one and the field is grey out in the UI.

Table 9 Scenario Properties

8.3.6.1 Workflow Categorization

Each Workflow within a Scenario is categorized as one of:

- Setup
- Run Test
- Compare
- Teardown

The UI displays Workflows within Scenarios grouped by these categories to help users understand the role of each Workflow. The category of a Workflow is set by the user when they add the Workflow to the Scenario.

8.3.6.2 Workflow Reuse

Each Scenario has a list of Workflows associated with it. When the Scenario is executed by the Test Case Runner each Workflow in the list is run in sequence unless parallel processing is selected. If a Workflow fails, execution of the Scenario is terminated, a failure is recorded, and subsequent Workflows are not executed.

Workflows that carry out sequences of Actions required by multiple Scenarios can be incorporated into the Workflow lists for more than one Scenario.

For example, a Workflow can be created to clear down a collection of tables. If many Scenarios all need the ability to clear down the same collection of tables, they can all share the same Workflow which performs this function. If the set of tables to be cleared changes, only one Workflow must be updated.

8.3.6.3 Parallel Processing

If the "*Run Parallel*" flag for the Scenario is set to true, all Test Cases within the Scenario will be processed in parallel subject to the "*Maximum Processes*" value.

8.3.7 Action

Actions are programmed in C# .NET within the ATS3 source code tree. Some are visible globally to all Departments and Applications. Others may be defined at the Department or

Application level. Actions defined at the Application level can implement functionality relevant to a specific application. For example, EPA can implement an Action "*Build Account*" which is irrelevant to other Applications. This Action will only be visible when configuring Workflows within the EPA Application.

The Actions that are visible when constructing Scenarios are:

- Global Actions
- Department Actions
- Actions relevant for the current Active Application (see section 9.1.1)

Examples of Actions are:

- Execute Shell Command
- Call SSIS Package
- Execute SQL Statement
- Call Stored Procedure (Specialisation of Execute SQL Statement which has parameters for the procedure name and parameters of the procedure)
- Truncate Table (Specialisation of Execute SQL Statement which only requires a table name)
- Purge Table (Specialisation of Execute SQL Statement which allows a subset of records to be deleted)
- Compare Table
- Count Records
- Build Account
- Build Performance
- Export Security

Different Actions have different Input Value requirements. As a result, each Action type has its own, dedicated configuration screen within the UI.

Actions make use of Input Values from two sources. Scenario Properties supply values Actions can use. Scenario Properties supply Input Values that are global and shared across all Test Cases within a Scenario. Test Cases also supply Input Values that Actions use.

For example, the Shell Command Action type requires a text string containing the command and perhaps a parameter to define success and fail return values. Where values within the shell command are different from one Test Case to another, each individual Test Case will supply the necessary values at the time the Action is executed for the Test Case.

Alternatively, the Scenario can supply an Input Value so the same value is used repeatedly across all Test Cases within the Scenario.

In the case of an Action that implements a file import using SSIS, the Action must have parameters defining the details of the SSIS package and these are configured in the Action's dedicated UI screen. The actual file to be imported is a property of the Test Case so that multiple Test Cases that import different files can be run by the same Action.

8.3.7.1 Parallel Processing

Actions have a static property called "Can Run Parallel." If this is true users can flag instances of the Action within Workflows to "*Run Parallel*." If this value is false, the UI greys out the option to "*Run Parallel*" when the Action is added to a Workflow.

8.3.7.2 Actions for Importing Files

The purpose of some Actions is to run a process to import a file into a database. The Input Values for Actions are supplied by Scenarios and Test Cases. When configuring a Scenario or a Test Case so it can supply a file to an Action that will perform the import, the file to be imported is uploaded into the ATS3 storage tier.

When the Action runs it will obtain the file to be imported from the ATS3 storage tier.

The file system and shared folders are NOT used by the ATS3. All files required by Actions are kept in the ATS3 storage tier.

8.3.7.3 Actions for Comparing Data

The purpose of some Actions is to compare Actual Results with Expected Results and write the outcome to the Compare Results Tables used by the Data Compare Tool and Test Results Report (see section 10 for details).

8.3.7.4 Actions for Event Driven Processing

Most applications have modules which are event driven. For example, EPA is architected around a collection of queues and daemons which monitor the queues. When jobs are added to the queues the appropriate daemon detects this, commences processing, and then reverts to monitoring the queue.

In ATS2 Test Cases trigger data processing and then wait for a specified delay measured in second or minutes. This causes two issues:

- If processing is completed before the delay times out, time is wasted waiting for the time out.
- If processing completes after the delay has completed ATS2 will begin performing data compares before Actual Results are ready to be compared. This is a frequent source of false breaks in ATS2.

In ATS3 it will be necessary to implement application specific Actions that can wait for application-level events to complete. Users will be able to incorporate Actions to trigger data processing and Actions to wait until the processing event is complete into Workflows. This will prevent time being wasted and will prevent data compares commencing before output is ready to be compared.

Actions designed to wait until an event takes place have a "Time Out" setting. This allows users to configure that amount of time after which the Action should give up waiting and report a Test Case failure.

8.3.7.5 Actions for Scenarios to Test Purging and Housekeeping

Scenarios will be created by users to verify purging and housekeeping functions. It will be necessary to implement Actions to support this class of Scenario. The steps to test a purge function are:

1. Count the records to be deleted.
2. Count the records that must not be deleted.
3. Call the purge function.
4. Count the records that should have been deleted. The result should be zero.

- Count the records that should not have been deleted. The number should match the value recorded in step 2.

Actions will need to be built to support this Workflow. This requires persistence of values read by the Actions in steps 1 and 2 so they can be used by steps 4 and 5.

8.3.8 Test Results

Test Results are created when one or more Scenarios are executed. Test Results provide input to the Test Results Report (section 10.4) and the Data Compare Viewer (section 10.2).

Name	Description
ID	An automatically generate ID number.
Run Type	Either "Baseline Run" or "Test Run".
Scenario	Reference to the Scenario the results relate to.
Creation Details	User, date, time.
Test Pack	Reference to the Test Pack the Test Case belongs to.

Table 10 Results Set Properties

Test Results (expected and actual) can be stored in Excel documents uploaded into the system and in database tables in the Expected Results environment. Where Test Results are stored is determined by the C# logic of the specific Action performing the data compare.

Actions performing data compares must always write Test Results to the Compare Results Tables (section 10.3) owned by the Data Compare Viewer (section 10.2).

8.3.9 Test Case

Test Cases are configured in the UI. Each Test Case has a reference to the Workflow it supplies Input Values to. The UI screen used to edit a Test Case must be designed to obtain the set of fields that must be populated from the Actions within the Workflows within the Scenario.

The Test Case fields are listed in the screen in the same sequence as the Actions of the underlying Workflows.

Name	Description
ID	An automatically generate ID number which is unique within the Application the Test Case belongs to.
Name	A short name for the Test Case.
Description	Description of the Test Case.
Creation Details	User, date, time.
Last Update	User, date, time.
Test Case Properties	The number and type of these values are dynamic and depend on the Actions within the Workflows of the Scenario the Test Case is for.
Test Pack	Reference to the Test Pack the Test Case belongs to.
Comment History	A collection of comments added by users. Each comment has a creation date / time / user stamp.
Tracking Keys	A collection of key / URL pairs. The key may contain Jira, ServiceNow, etc. keys and the URL is a link to the ticket in the associated tracking platform.

Table 11 Test Case Properties

The QA teams for PMS, EPA1, and EPA2 use the following strategy for creating set up data before running tests:

1. Truncate accounting data tables
2. Import transactions
3. Build accounts to create positions and lots

This avoids the need to import positions and lots. All Test Cases for these applications supply transactions, but not positions or lots.

8.4 File Import Example

The Workflow "*Import File*" will define the Actions to perform.

1. Truncate staging table.
2. Call SSIS to import file.
3. Count imported records.
4. Compare imported records to expected results.

A Scenario of type "*Import File*" will then provide a list of Test Cases to run. If the Scenario is called "Import Price File" then the Test Cases within the Scenario could be:

- Price File 9th June 2025
- Price File 10th June 2025
- Price File 11th June 2025

A Scenario Property will supply the name of the staging table the files are going to be imported into. This is so the actions to truncate the table, count the imported records, and compare the results can all use the same Input Value containing the table name.

The Scenario will also supply a global property for the name of the SSIS package to be run. This is so all Test Cases inherit the same SSIS package name.

Each of the Test Cases will supply an Input Value giving the name of the file to be imported.

8.5 Indexing System

To help navigate the system UI and support communication in meetings and through Jira, Slack, e-mail, the Wiki, etc. an Indexing System is necessary to identify Test Cases and breaks. All screens in the UI and reports will display data using the Indexing System.

Scenarios must be allocated an integer ID number when they are created. The ID number must be unique within the Feature the Scenario belongs to.

Test Cases must be allocated an integer ID number which will also be unique within the Feature the Test Case belongs to.

Breaks must be allocated an integer ID number which is unique within the Application the break belongs to.

Indexed keys for test cases take the form:

App-Component-Feature-Scenario ID.Test Case ID

Examples:

EPA-UI-TXN-123.456

EPA-DB-POSN-789.123

EPE-App-DP-987.654

Indexed keys for breaks have the same format as indexed keys for test cases and have a hyphen plus the unique ID of the break appended.

9 User Interface

This section provides the design for the system user interface. The user interface will be web-based, implemented in ASP.NET.

9.1 Concepts

9.1.1 Active Application

In the common use case ATS3 users will focus on an individual application. A menu option will allow the user to select the *active application* from a tree hierarchy containing all available applications grouped by department.

When the active application has been selected all other UI components will display data related to the active application and enable interaction with test scenarios and test cases associated with that application.

9.1.2 System Heading

All UI screens will have a left-justified heading at the top. The heading will have the following format:

Department / Application / Component / Feature / Screen Name

Department will contain the department the active application belongs to. *Application* will contain the name of the active application (see section 9.1.1). *Screen Name* will be the name of the current screen being displayed.

9.1.3 Colours

Through the entire UI everything displayed relating to tests being successful and having passed will be drawn using the colour green. Everything relating to tests having failed will be drawn using the colour red.

9.2 Data Deletion

Some parts of the UI allow data to be deleted. At the storage level no data will ever be deleted. When users "*delete*" something the content will be marked as deleted in the database but will not actually be removed.

Therefore, all tables storing items such as Departments, Applications, Components, Workflows, Scenarios, Test Cases, etc. must have a field to indicate whether the record is active or deleted.

9.3 Roles

The UI supports three roles.

Name	Description
All Users	Any user with the ability to access the ATS3 UI.
Testers	Members of the BA, QA, and development teams responsible for designing, building, and running tests.
Managers	Team members with permission to make high impact changes such as editing Departments, Applications, and Components, and deleting content.

Table 12 User Interface Roles

9.4 Sections

The user interface will comprise the following key sections:

- Application Manager
- Environment Manager

- Test Pack Manager
- Test Run Manager
- Test Case Manager
- Testing Dashboard
- Data Compare Viewer (see section 10.2 for details)

9.4.1 Application Manager

The Application Manager allows users to create, edit, and delete Departments, Applications, Components, and Features.

Departments can only be deleted if they contain no Applications. If there are Applications associated with a Department the option to delete it must be greyed out. When Applications and Departments are deleted all their Components, Features, Scenarios and Test Cases are also automatically deleted.

9.4.1.1 Permissions

All users have permission to view all Departments, Applications, and Components. Only QA Managers have permission to create, change, and delete content.

9.4.2 Environment Manager

The environment manager allows users to view the status of environments relating to the active application and manage them. Environment management activities include:

- Adding, editing, and removing environments.
- Refreshing data from an upstream environment.
- Installing a Fix Version.

9.4.2.1 Environments Description

Each testable application must have a series of environments to enable automated testing to function. Each test environment has a specific purpose. Table 13 provides details of the environments each application must have.

Sequence	Name	Description
1	Production	The production environment hosts the version of the application currently in use by the business.
2	Landing Pad	Data and code refreshes from production to non-production environments must be managed and owned DBA's and system administrators. Data and code are copied automatically once every 24 hours from the production environment to the landing pad environment. This is handled by jobs owned by system administrators. The Landing Pad allows control of refreshes further downstream to be owned by application teams.
3	Baseline	The Baseline Environment contains data and code used to refresh Test Environments when preparing to run tests. The Baseline Environment is refreshed on-demand from the Landing Pad by the testing team using the Environment Manager.
4	Test	The Test Environment is where test cases are executed to generate results.
5	Expected Results	The Expected Results environment contains output following execution of Test Cases against the current production Fix Version of the application under test. This environment is refreshed from the Test Environment by the test team using the Environment Manager on demand after running test cases against the production Fix Version.

Table 13 Environments Definition

The Environment Manager must make it impossible to copy results to the Expected Results environment if the results in the Test Environment have not been generated using the Production version of the application.

Test environments are grouped into *Test Instances* so multiple regression tests can be carried out in parallel. The only environment which exists outside of a Test Instance is the *Landing Pad*.

9.4.2.2 Environments Display

The UI will show a diagram of environments. The Landing Pad is displayed first. Then the other environments are displayed below, grouped by Test Instance. Each environment will be displayed with the date / time / user of the most recent data refresh and the currently installed Fix Version along with the date / time / user relating to when the Fix Version was installed.

9.4.2.3 Environment Properties

Each environment has properties which can be edited through the UI.

Name	Description
Classification	Defines the type of the environment (Production, Landing Pad, Baseline, Test, Expected Results, Etc.)
Name	A short name for the environment which will be displayed on screen and in reports.
Application	A reference to the application the environment is used for.
Hostname	Name of the host server for the environment.
Scale	This defaults to "Full" for all environments. For Test and Expected Results environments it is permitted to change the value to "Subset" to indicate the environment does not contain a full data set.

Table 14 Environment Properties

9.4.2.4 Management Commands

The following commands must be implemented as either buttons, menus, or a combination. Commands relating to specific environments (EG “Edit” and “Delete”) should be grouped with the environment they relate to either in the menu system or the environment diagram.

Name	Access	Description
Add New	Managers	Allows a new environment to be defined.
Edit Existing	Managers	Allows the properties of the environment to be altered.
Delete Existing	Managers	Allows the environment definition to be deleted.
Refresh Data	All Users	Causes the environment to have its data refreshed from the upstream environment.
Install Fix Version	All Users	Allows a specific Fix Version to be installed to the environment.

Table 15 Environment Management Commands

9.4.2.5 Environment Sizing

The common use case for the system is executing small sub-sets of Scenarios or Test Cases. Most test runs do not require a full environment with a full set of data. Most test runs do not require data going back more than several months.

Full application regression tests are less common and take place several times per month.

The environments of types Test and Expected Results must be defined as either “Full” or “Subset”. A solution must be devised to support:

- Fast refreshes of a limited data set:
 - A small number of accounts
 - Two months data history

- Full refreshes comprising the complete data set:
 - All accounts
 - All data history

9.4.3 Test Pack Manager

Test Packs contain Scenarios, Test Cases, and Test Results related to individual Application versions. The Test Pack Manager screen displays Test Pack information for the Active Application.

Test Packs are displayed in a list. Users may sort the list by creation date, creation user, Fix Version, code branch, etc. Users may display and hide Test Packs that have been archived.

The following user actions must be supported:

- **Create New** – Allow user to enter details for a new, empty Test Pack. The name, Fix Version, and Code Branch details of the new Test Pack must be unique.
- **Copy** – Allow user to copy an existing Test Pack. The name, Fix Version, and Code Branch details of the new copy must be unique.
- **Edit** – Allow user to edit the properties of Test Packs.
- **Archive** – Mark Test Packs as Archived.

9.4.4 Test Run Manager

The Test Run Manager allows users to start, pause, and terminate test runs. When test runs are in progress the Test Run Manager allows all users to view status of the run which is updated on-screen in real-time.

9.4.4.1 Test Case Selection

The Test Run Manager allows users to select a collection of tests to include in a Baseline Run or Test Run using checkboxes. The UI displays a tree view containing Departments, Applications, Components, Features, Scenarios, and Test Cases. There is a checkbox next to each item in the tree allowing it to be selected and deselected.

9.4.4.2 Run Type Selection

The Test Run Manager automatically determines whether the run will be a Baseline Run or a Test Run. It achieves this by interrogating the version of the application installed in the Test Environment and comparing it to the current production version of the application. The phrase "Baseline Run" or "Test Run" is displayed prominently at the top of the screen. Colour coding is used within the screen to make it clear which type of run the Test Environment is ready for.

9.4.4.3 Volume Control

An important feature of ATS3 is to give users the ability to control the volume of data used in Test Runs.

When an application version is immature the number of defects and breaks will be higher. It will be sufficient to execute low volume Test Runs which complete and provide results quickly. Running full volume tests on immature application versions wastes time.

As an application version becomes more mature a low volume Test Run will not exercise enough variations to discover all defects. The Test Run Manager will allow users to increase the volume of Test Runs to find more defects. Increasing volume will also increase run-times for Test Runs.

The final Test Run of a mature application version immediately prior to deployment to production must be a full volume Test Run. Such Test Runs will take longer to execute.

Volume Control is not permitted for Baseline Runs. Baseline Runs will always execute using full Test Case and data volume. If Expected Results need to be regenerated this can be done quickly by selecting only the relevant Scenarios or Test Cases for the Baseline Run.

9.4.4.3.1 Volume Control Templates

Managers can create and configure Volume Control Templates. Examples of templates Managers might choose to create are:

- **Quick Test:** Minimum Test Case and data volume so Test Runs are fast and obvious defects can be found quickly, fixed, and retested.
- **Standard Test:** Test Case and data volume is more realistic with respect to production data processing. The duration of a Test Run will be longer, but results will be more conclusive.

- **Full Test:** Full regression Test Run with 100% Test Case and data volume.

When users prepare for a Test Run, they can select the volume they wish to run from the list of templates Managers have created.

9.4.4.3.2 Test Case Volume

When creating and editing Volume Control Templates the Test Case Manager UI will allow the user to select the number of Test Cases in each Scenario to run. This value is selected at the Test Run level and is applied to all Scenarios selected for execution in the Test Run.

Users will be able to enter a specific number of Test Cases to execute or a percentage. If the percentage is low and results in a value less than one for some Scenarios, then the value will be rounded up to one so at least one Test Case is executed for each Scenario.

9.4.4.3.3 Data Volume

The C# interface all Actions implement will provide the ability to control data volume. ATS3 will be able to tell Actions the number of records to act on or a percentage of records to act on. It will also be possible to constrain the number of accounts included in a Test Run and the date range of data used. Each Action can define its own set of volume constraints.

For example, Actions that import records or compare records will have the ability to limit the number of records imported and compared.

When creating and editing Volume Control Templates the Test Case Manager UI will allow the user to select the number of records or percentage of records to be used by Actions that will be executed during the Test Run.

The set of data volume controls displayed by the UI in the template editor dialogue is generated dynamically at run-time. Its contents depend on the constraints supported by all the Actions included in Workflows used by Scenarios defined for the application.

9.4.4.4 Real-Time Status View

When the user clicks the button to start a Test Run the Test Run begins. While the test is running the UI updates the status of each item within the tree view in real-time.

Statuses that are displayed are:

- Not Started
- Running
- Complete – Passed
- Complete – Failed

Statistics are displayed at the top of the screen and are updated in real-time as progress through the run is made.

- Applications, Components, Features, Scenarios, Test Cases – Not started, running, complete – passed, complete failed.
- Counts and percentages.

Green and red are used for passes and fails.

When a run is completed the Test Results Report screen is displayed. See section 10.4 for details.

9.4.4.5 Logging

When a Run is processed by the system the start time, end time, duration, and outcome are written to a log. Log messages must all contain the index key for the Test Case they relate to. This is so hyperlinks can be inserted into the Test Results Report that link log messages to the results they relate to.

The log can be viewed in real-time, on screen while a Run is taking place.

9.4.4.6 Test Results Report

Upon completion of a Test Run a Test Results Report is generated. The report documents all breaks and includes sections describing Tolerances and Acceptable Differences that have been applied to each Data Compare Action. See section 10.4 for more information.

9.4.5 Test Case Manager

The Test Case Manager allows users to create and configure Scenarios. Users can edit the sequence of Actions within Workflows and associate Workflows with Scenarios. They can add Test Cases to supply Input Values to Actions. They can set global Input Values by assigning them to Scenarios.

When editing Test Cases users can supply SQL statements, filenames, dates, accounts, and other values which Actions will read.

Some types of Action may allow Excel documents and text files to be submitted as properties of Scenarios and Test Cases.

9.4.5.1 High Volume Input Values

Some Actions require Test Cases to supply large volumes of data stored in pre-prepared tables. For example, some Actions will call stored procedures that require table value parameters containing many records.

The Test Case Manager allows users to import records from text files, Excel documents, and other databases and store those records in the ATS3 database to be used as Input Values for Actions. A dynamic storage system must be designed and built in the ATS3 database so developers are not required to create tables necessary to hold high-volume Input Values.

9.4.5.2 Inline Documentation

Scenarios and Test Cases have comment histories and a list of tracking key / URL pairs. Users can add comments to Scenarios and Test Cases. Each comment has a creation date / time / user stamp. Comments are shown in the display with Scenarios and Test Cases. They may be ordered by date / time ascending and descending. Tracking key / URL pairs are also shown in the display for Scenarios and Test Cases.

Comments are carried over from one Test Pack to another. When a Test Pack is copied, the new copy will retain all comment history from the source Test Pack.

9.4.5.3 SQL Editor

In ATS2 many SQL statements used by Test Cases are stored outside the application in text files in the source repository and on file shares. This contributes to the system's complexity.

In ATS3 all SQL statements will be created, edited, and managed within the system UI. Separate, external text files will not be supported.

The ATS3 UI allows global Input Parameters to be assigned to Scenarios, and some parameters may be SQL statements. The UI allows parameters for Test Cases to be edited and many Actions that Test Cases supply Input Values to require SQL statements.

To limit the need to switch between the ATS3 UI and a SQL client when designing, prototyping, and testing SQL statements the ATS3 UI will have a reusable SQL statement editor dialogue. This will allow users to perform tasks similar to those supported by full SQL clients. The goal is not to replicate the features of a full SQL client, but to provide a small number of common features to speed up the process of creating and editing SQL statements within the ATS3 UI.

Some features the ATS3 SQL statement editor dialogue will support are:

- Ability to enter and edit SQL statements including long and complex statements.
- Ability to submit statements to any of the Active Application's environments and view results.

9.4.5.4 Scenario Debugger

The Test Case Manager allows users to step through the Actions in Workflows associated with Scenarios, Action by Action. The UI allows the outcome of each Action and any in-memory values it creates to be viewed in-situ.

9.4.5.4.1 Stepping

When stepping through a Scenario users can dynamically select any Action as the next Action. Or by selecting the "Step" option the debugger will automatically step to the next Action in the sequence.

Users can stop the stepping process.

9.4.5.4.2 In-Memory Values

Some Actions create run-time values in memory which are passed from one Action to the subsequent Actions in the Workflow. These values behave like the values of variables in regular software programming.

Values generated by an Action have the same purpose as Input Values supplied by Scenarios and Test Cases. Therefore, a common architecture should be designed to handle Input Values supplied by Scenarios and Test Cases and those generated as output by Actions.

The Scenario Debugger will require the ability to view and change in-memory values created by Actions which are visible to subsequent Actions. It will also allow Input Values supplied by Scenarios and Test Cases to be edited. See section 8.3.7.5 for an example of where this functionality will be required.

Important: In-memory values are necessary to make Workflows such as the one described in section 8.3.7.5 possible. They are not intended to allow Scenarios and Workflows to start to be “programmed” like a regular software language.

In-memory values are local to Workflows. In-memory values generated in a Workflow are discarded when the Workflow completes and are not passed to any subsequent Workflows in the Scenario. The purpose of this is to limit the complexity of Scenarios and avoid dependencies between Workflows.

Actions are not permitted to change in-memory values passed to them by Actions which took place earlier in the Workflow. The purpose of this is to limit the complexity of Scenarios.

9.4.6 Testing Dashboard

The Testing Dashboard allows users to view the status of testing for the active application and historical track record of testing statistics. Examples of information displayed are:

- Fix Version installed in each environment.
- Environments where activity is currently taking place, for example environment refreshes and test execution.
- Component / Feature / Scenario Test Status
 - A list of named Components and Features for the active application with each one marked as passed or failed. The total number of Test Cases for each Component and Feature will be included alongside its name as well as the percentage of passes and fails.

- Nested below each Component name in a tree will be the Features and Scenarios within the Component. Each row will show the Scenario name, number of Test Cases passed and failed, and percentages.
- Test Run History
 - A log of test runs showing which user initiated the test run and the date / time the test run started and finished.
 - Total test cases passed and failed with percentages.
- Test Results History
 - The Component / Feature / Scenario Test Status section of the dashboard shows results for the current test run. The Test Results History section shows results in a chart with number of Test Cases in the y-axis and date in the x-axis.
 - The purpose of the chart is to allow users to see whether the number of fails is increasing, stable, or decreasing over time.
 - Each bar in the chart is stacked. The lower part of the bar is green and represents the number of Test Cases passed. The upper part of the bar is red and represents the number of Test Cases failed.
 - The y-axis scales automatically.
 - The user can select the date range for the x-axis:
 - Today
 - 1 Week
 - 1 Month
 - 3 Months
 - 6 Months
 - 1 Year
 - 2 Years
 - 3 Years



- Custom Date Range

The Testing Dashboard shows a list of all Test Results that have not been archived with summary level values. Users can drill down into the list to jump to the Test Results Report for each Test Run. This allows users to access Test Results Reports for past Test Runs.

10 Data Comparison

Comparing Actual Results with Expected Results is a fundamental aspect of testing activity. This section describes system features that support data comparison.

10.1 Data Compare Actions

Different Action types are programmed within the system in C#. Actions are assembled into sequences called Workflows and Workflows are assigned to Scenarios. Reusable Actions will be created that implement different types of data comparison, for example:

- Record Compare
- Table Compare
- Record Count
- File Compare

When a Data Compare Action is executed it will perform its specific comparison logic making use of Input Values supplied by the Scenario, Test Case, and Actions earlier in the Workflow. The Scenario or Test Case will supply things such as the key of the record to compare, the table name to be compared, or the table to count records in. Actions can also generate similar values for use as input to Data Compare Actions.

Data Compare Actions write their results to a set of standard tables in the system storage tier. These tables are owned by the Data Compare Viewer.

Some Data Compare Actions will allow users to configure fields to ignore, for example record ID fields and timestamp fields. The details of fields to ignore will be supplied by the Scenario and / or the Test Case.

Important: Data Compare Actions do NOT know about Tolerances and Acceptable Differences. They do NOT ignore fields such as ID's and timestamps. Whenever a Data Compare Action runs it will report all real differences by writing them to the results tables owned by the Data Compare Viewer. It is the Data Compare Viewer that applies Tolerances and Acceptable Differences to the results generated by Data Compare Actions. The Data Compare Viewer knows which fields to ignore.

10.1.1 File Compare Actions

Some Scenarios are intended to verify the creation of exported files. Data Compare Actions capable of comparing an actual file with an expected file will be required.

Fields to be ignored can be provided as Input Values by Scenarios and Test Cases.

The expected files used by File Compare Actions are stored within the ATS storage tier, not the file system, source repository, or shared folders.

Actual files generated during Test Runs will be saved into the Compare Results Tables used by the Data Compare Viewer. This is so they can be called up and displayed when users are investigating breaks.

10.1.1.1 Architecture

A base class that implements file comparison logic will be written. Child classes will be derived from it that support comparison of different file formats (XML, CSV, Excel, pipe delimited, custom, etc.)

10.2 Data Compare Viewer (DCV)

The Data Compare Viewer (DCV) is part of the system UI. It allows users to view results generated by Data Compare Actions in a standard format. The DCV implements many features to improve the productivity of the application team.

- Matches are displayed in green.
- Breaks are displayed in red.
- Near misses, when values are almost the same within a tolerance are displayed in orange.
- Matching records can be hidden by the user.
- The number of records displayed can be constrained to speed up analysis early in the test life cycle when the number of breaks is high.
- Users can add comments to breaks and view comment history.

- Statistics are displayed at the top of the screen showing the number of matches and breaks at the record and field level. The number of singletons (expected and actual) with percentages is also displayed.
- Users can hover over differences to view a tooltip showing the amount of the difference or delta.

The DCV displays entire records one per row. Fields run left to right in columns. Expected and actual results for each field are shown adjacent to each other. Fields can be sorted by name and data type.

Expected singletons are displayed first. Then actual singletons are listed. Then non-singleton rows are displayed. The UI allows singletons to be hidden. The UI allows matching records to be hidden.

Where a field matches the background colour is green and the text colour is white. Where a field matches due to a Tolerance or Acceptable Difference that has been added the background is white and the text colour is green. Where a field does not match the background is red and the text colour is white. Where a field nearly matches and is within a tolerance the background is orange, and the text is white.

Colours are configurable within the database, but users do not have the ability to change colours. This is so all team members screens and reports correspond to each other.

10.2.1 Debugging

When investigating Scenario fails team members often need to determine whether the cause of the break is the application being tested or is the Scenario, Workflow, Test Cases, or the Input Values. When applications are enhanced and fixed, these changes often cause tests which previously passed to start to break. The solution is to fix the tests so they match the new desired behaviour of the application.

It must be easy to jump from the DCV to the Test Scenario Debugger. Users can then quickly determine the root cause of the break.

10.2.2 Tolerances and Acceptable Differences

There are two types of tolerance.

- **Sign-Off Tolerance:** When fields match within the sign-off tolerance the field is displayed in green, and the values are assumed to be close enough to be signed off. Sign-off tolerances are listed in the Test Results Report (TRR) – see section 10.4 for more information.
- **Near Miss Tolerance:** Near miss tolerances causes fields in the DCV to be displayed using orange. This is to help users spot values that were close, but not close enough to be signed off. Near miss tolerances do not result in a sign-off and near miss fields constitute a data break. Near miss tolerances are not listed in the Test Results Report.

Users can add sign-off tolerances to clear breaks. All tolerances track the creation user, date, and time. Sign-off tolerances also have a mandatory field to store the reason for the tolerance. The reason can be selected from a list of pre-configured reasons. Custom reasons can also be entered manually.

Signoff tolerances added in the DCV are included in Test Results Reports.

Individual acceptable differences can be added at the field and record level. Adding an acceptable difference clears the break. Acceptable differences track the creation user, date, and time and have a mandatory field to store the reason for the acceptable difference. Acceptable Differences added in the DCV are included in Test Results Reports.

Important: Actual and expected results in the results tables are never edited or changed. The results tables always stored the real output of the baseline and test runs. Tolerances and Acceptable Differences are applied to values read from the results table when the DCV renders results and when Test Results Reports are generated. Tolerances and Acceptable Differences are overlaid on to results when results are read. They are NOT used to change results.

10.2.3 Break Reporting

The DCV allows users to report breaks and assign them to other team members. When the user completes assessment of the break, they open a dialogue to create a Break Report.

Breaks can be reported at the Scenario level and the Test Case level. Breaks are assigned a cause and routing information.

When a break report is created it is logged in ATS3. The user may also select a tick box which will cause a Jira ticket to be created and assigned to the appropriate person depending on the chosen routing.

Break causes are configured through the UI by Managers. A global set of default causes are defined and cannot be changed. At the Department and Application level Managers can choose which of the global causes are visible in the Break Report dialogue. They can also add additional causes visible locally within Departments and Applications.

Note: Some of the functionality described below duplicates features already in Jira. Due to this crossover several options under discussion are:

1. Stop using Jira for management of breaks and fixes.
2. Track breaks and fixes in ATS3 and Jira with integration to keep Break Reports and Jira tickets synchronised.
3. Do not track breaks in ATS3 and rely solely on Jira.

There are various pros and cons to each of these options.

Field Name	Description
Level	Scenario or Test Case
Key	The indexed key of the Scenario or Test Case that has failed with the unique ID for the break appended.
Creation Stamp	Date, time, and user.
Expected	Value generated by a baseline run.
Actual	Value generated by a test run.
PK Expected	Primary key used for record matching.
PK Actual	Primary key used for record matching.
Comments	Explanation supplied by the person reporting the break. Additional comments may be added throughout the life of the break.
Cause	Selected from a configurable list: Outdated Test Case, Outdated Input Values, Outdated Expected Results, Data Integrity, Application Defect
External Ticket Created	Indicated whether a Jira (or other) ticket has been generated for the break.
Routing	Where to send the report. This can be QA Team, DV Team, BA Team, or an individual person. This will impact who the resulting Jira ticket is assigned to.
Status	From a configurable set of values (New, Analysis, Waiting for Fix, Retest, Resolved, Failed, Closed, Abandoned).

Table 16 Break Report Definition

10.2.4 Break List Viewer

The Break Report List Viewer (BLV) is a UI screen which lists Break Reports. Break Reports can be sorted and filtered by Department, Application, Component, Feature, Scenario, Test Case, creation date / time, creation user, last update date / time, and last update user.

Each row in the BLV includes the Indexed Key for the break.

Hyperlinks allow users to jump from rows in the list to all the associated entities.

Rows indicate whether a Jira ticket has been created for the break. If a Jira ticket has not been created the user can press a button to cause a Jira ticket for the break to be created and routed (assigned).

The BLV allows users to add comments to breaks, assign them, and change their status.

10.2.4.1 Resolution Via Acceptable Difference or Tolerance

Sometimes the correct resolution of a break is to add an Acceptable Difference or Tolerance. The UI will provide functions at the break level to resolve breaks by creating either a new Acceptable Difference or a new Tolerance.

10.3 Compare Results Tables

The compare results tables are owned by the DCV. A standard set of tables are provided which Data Compare Actions write actual and expected results into. There are multiple results tables, one per data type (integer, string, decimal, date/time, etc.) Table 14 below shows the fields each results table has.

Field Name	Description
Source Name	Name of the table or file the result comes from.
Field Name	Name of the field the result is for.
Expected	Value generated by a baseline run.
Actual	Value generated by a test run.
Test Run	Reference to the baseline / test run that generated the result.
PK Expected	Primary key used for record matching.
PK Actual	Primary key used for record matching.

Table 17 Compare Results Table Definition

When the result being written to the results tables is for a record count the results table for (long) integers will be used and the Field Name will be "Record Count."

10.3.1 Beyond Compare

The QA team currently use a third-party tool called Beyond Compare. The DCV will be more powerful and easier to use than Beyond Compare. Useful features within Beyond Compare can be replicated by the DCV.

10.4 Test Results Report (TRR)

The Test Results Report (TRR) is a report displayed online within the UI. It supports drill-down into results and stepping between results and log messages using hyperlinks. Users can click on rows for compares of individual Scenarios to open the DCV and view the detail level results of the compare.

The TRR can be exported to Excel and PDF.

In the user interface the TRR hides matches and successful results by default. A button is provided to unhide them.

The TRR follows the system standard colours (green for passes, red for fails / breaks, orange for near misses).

The TRR makes use of the values in the results tables, Tolerances, and Acceptable Differences in a similar way to the DCV. The DCV displays results generated by a single Data Compare Action. The TRR contains results for all Data Compare Actions in an entire Test Run.

10.4.1 Summary Section

The TRR starts with a summary section providing an overview and statistics:

- Department.
- Application.
- Test Run volume information. If the volume of Test Cases or data were less than 100% then a warning message is displayed prominently in the report.
 - Percentage Test Cases run.
 - Any data volume constraints applied to the Test Run.
- List of Components tested. The report indicates whether the Component was included in the Test Run or was excluded.
- Test run start time, end time, duration, username.
- Number of Scenarios run. Number passed and failed, and percentage.
- Number of Test Cases run. Number passed and failed, and percentage.
- Number of Workflows run.
- Number of Actions run.
- Number of sign-off Tolerances applied.
- Number of Acceptable Differences applied.

10.4.2 Tolerances and Acceptable Differences

The TRR then lists out all sign-off Tolerances and Acceptable Differences including the name of the Component and Scenario they belong to, their creation date, time, and user and the justification comment for each one.

The purpose of this section is to support final signoffs before deployment to production.

10.4.3 Test Results

The TRR then iterates through all Components and Features that were included in the Test Run. For each Component and Feature a table is created showing all the Scenarios that were run and whether they passed or failed.

For failed Scenarios the number of singletons, record matches / breaks, field matches / breaks, and percentages are shown.

Hyperlinks to the relevant lines in the Log section of the report are included.

10.4.4 Log

After the Test Results section the TRR has a section for the system log. Messages from the log which relate to the Run are listed in start date / time sequence. Each log message contains a hyperlink that allows the user to step to the relevant part of the Test Results section.

11 Architecture

This section provides details of some architectural considerations.

11.1 File / Document Storage

As described in section 7.5 all files and documents used by the system will be stored within the storage tier (database) and not on external file shares or in source code repositories.

The ATS3 database will have a single Documents table to store details of all input files and documents. The various parts of the application that require files will use references to records within the Documents table.

Field Name	Description
ID	Primary key.
Name	Short name for the document.
Filename	Base filename of the document.
Description	Optional.
Creation Date / Time	Set when file first uploaded.
Creation User	Name of person who uploaded the first version.
Last Update Date / Time	Set each time document replaced with new version.
Last Update User	Set each time document replaced with new version.
Data	BLOB containing the file.
Archived	Boolean yes/no. Default no.
Version	Counter that is incremented each time the file is updated. This allows the system to retain prior versions of documents.

Table 18 Documents Table Definition

11.2 Excel Document Format

ATS3 will only use Excel documents to supply records for use as setup data, test input data, and expected results. All Excel documents must use the same format. The ATS3 UI will validate Excel files when users attempt to import them and will prevent badly formatted documents from being imported and used within the system.

The goal of enforcing a single document design is to reduce complexity and support creation of a standard, reusable toolset.

- Filename – Any filename can be used. Filenames are no longer relevant once documents have been imported because documents are associated with the purpose they will serve at the time they are imported.
- Worksheets – Only the first worksheet may contain values. If more than one worksheet is present all worksheets other than the primary must be empty.
- Cell A1 contains the acronym of the application the document relates to (EPE, EPA1, EPA2, EPM, ESRM). The UI importer validates this cell contains a recognised value.
- Cell B1 contains the name of the entity for the record (Transaction, Lot, Position, Price, Security, Account, etc.) The UI importer validates this cell contains a recognised value. The list of supported values is configured through the UI by QA Managers at the application level.
- Row 2 contains a sequence of field names. The sequence of the fields is not validated and documents for the same entity are permitted to use different collections of fields in different sequences depending on their specific needs. The first column, A, must contain a mandatory field. Columns must be contiguous – a blank column signifies the end of the row, and no further columns will be used.
- Rows 3 onwards contains data records. Rows must be contiguous – a blank value in column A signifies the end of the data set.

12 System Usage

This section provides information on how the system will be used.

12.1 Key Cycles

The system is used for the following types of cycle:

- Baseline Refresh Cycle
- Expected Results Creation Cycle
- Application Verification Cycle

12.1.1 Baseline Refresh Cycle

The Landing Pad environment is refreshed from Production daily. Periodically team members will use the system to copy a full data set from the Landing Pad environment to the Baseline environment.

The Baseline Refresh Cycle is typically performed following deployment of a new version of the application to production.

12.1.2 Expected Results Creation Cycle

The Test environment is refreshed from the Baseline environment. The refresh will be either a full or a subset depending on the set of Scenarios to be run.

The current production version of the application is installed to the Test environment. Test Scenarios are then executed to generate Test Results.

The Test Results are copied to the Expected Results environment.

The Expected Results Creation Cycle is typically performed following deployment of a new version of the application to production, and must be performed after a Baseline Refresh Cycle.

12.1.3 Application Verification Cycle

The Test environment is refreshed from the Baseline environment. The refresh will be either a full or a subset depending on the set of Scenarios to be run.

The version of the application to be tested is installed to the Test environment. Test Scenarios are then executed to generate Test Results.

The Test Results are compared against results in the Expected Results environment.

The Application Verification Cycle is run whenever new code changes are available for testing, or following fixes to Scenarios, Workflows, or Test Cases.

12.2 Scenario Design

This section highlights issues with test case design in ATS2 and how to address them in ATS3.

12.2.1 Test Case Dependencies

Test cases have been built in ATS2 that depend on each other. Some collections of Scenarios must be run in the correct sequence as one Scenario creates Baseline Data for the next Scenario. This design prevents some individual Scenarios being run in isolation.

This design will not be used in ATS3. All Scenarios must be designed to be completely standalone and independent. This will make it possible to run all Scenarios individually and will increase flexibility.

12.2.2 Reliance on Baseline Data

Instances of Test Cases becoming stale after environment refreshes and reporting false breaks are common in ATS2. The cause of this is the test cases do not provide sufficient setup data.

For example, many EPA Test Cases provide transactions which are used to create positions and lots before the Test Case is run. However, following a full refresh of the baseline data the test cases cease to function correctly because the refresh has changed values the Test Case depends on. Full refreshes can change account and security flags, data used in payments calculations, amortization data, etc. When the Test Case is rerun, setup is achieved by

importing the Test Case transactions, but expected results no longer match actual results because something else has been changed when the baseline was refreshed.

The correct solution to this is to ensure Test Cases import all records impacting the test as part of their setup. By taking this approach, Test Cases will be completely standalone and should pass even after the baseline data have been refreshed.

12.3 Full Application Regression Tests

In ATS2 full scale regression tests are a distinct concept. These tests typically simulate the overnight data processing cycle and can repeat the cycle for a series of contiguous days.

In ATS3 full scale regression tests are implemented as Scenarios. A Scenario which implements a full-scale regression test of an overnight processing cycle will likely be composed of multiple reusable Workflows for setup, testing, data compares, and teardown.

Volume Control Templates configured for use with Scenarios that simulate overnight processing cycles will control the number of accounts processed by each Test Run.

The number of days the Scenario runs for will be set by adding multiple Test Cases, one for each day to be processed. The Volume Control Template can then control the number of days processing to run for by constraining the number of Test Cases that will be executed during a Test Run.

13 Continuous Integration / Continuous Delivery (CI / DD)

Transitioning to a CI / CD process is a priority for Operations Technology. ATS3 will be a key part of this strategy.

To support Continuous Integration, it will be necessary to provide integration with DevOps and / or Git Hub Actions. It must be possible to configure hooks so Test Cases can be triggered automatically at the Component and Feature levels when code is merged to specific code branches.

Continuous Delivery will only ever be supported up to and including Pre-Production Environments. We will never automatically deploy changes to the production environment.

Further discussion is required to decide how ATS3 will be applied in the context of CI / CD.

14 Test Case Migration

Many Test Cases have been created in ATS2. For example, EPADB has 248 individual Test Cases comprising 3,097 steps (as at June 2025). Ideally a solution can be devised to automatically transfer the Test Cases in ATS2 to ATS3 or assist with this process.

Tools to support Test Case migration will be discussed once a working version of ATS3 is operational.

There are some large challenges to solving this:

- Existing Test Cases have a complex structure.
- Many Test Cases which follow the ATS2 framework may not be directly translatable into the ATS3 framework of Scenarios, Workflows, Actions, and Test Cases.
- The various pieces of existing Test Cases are spread between Excel documents, worksheets within those documents, text files, file shares, code repositories, and databases.
- The structure of these pieces can vary from Test Case to Test Case.
- Cells within Excel documents contain key words and values using a bespoke syntax. It will be necessary to reuse ATS2 code to parse this syntax.
- Not all Test Cases function correctly, and their defects will be ported to ATS3.

It is possible the most productive and reliable way to transfer Test Cases may be to do it manually.

15 Project Plan

15.1 Approach

The project will be delivered in an agile, or *agile inspired* manner. The goal is to complete each part of the system 100% and sign-off before moving onto the building the next part.

15.2 Structure

The project consists of a series of phases. Each phase is split into a sequence of stages.

- Phase 1: Organisation User Interface
- Phase 2: Environment Manager
- Phase 3: Test Configuration User Interface
- Phase 4: Results Generator
- Phase 5: Test Runner
- Phase 6: Results Manager

15.2.1 Phase 1 – Organisation User Interface

The Organisation User Interface allows users to manage the structures used to group test related configuration.

Stage	Name	Description
1.1	Environment Acquisition	Obtain development servers for C#, IIS, SQL Server, and set up new GIT repository.
1.2	UI Layout	First runnable UI. Tabs on left for main functions. Menu at top for infrequently used functions. All place-holders – no next level screens at this stage.
1.3	System Heading	Add “breadcrumbs” system heading to UI and finalise UI layout.
1.4	Roles	Set up roles management. UI knows which role(s) current user is a member of and can make options active or greyed out accordingly. Users can be added to and removed from roles.
1.5	Application Manager UI – Departments	Add functions to create, edit, and archive / delete Departments.
1.6	Application Manager UI – Applications	Add functions to create, edit, and archive / delete Applications.
1.7	Active Application	Add function allowing users to select the currently Active Application.
1.8	Application Manager UI – Components	Add functions to create, edit, and archive / delete Components.
1.9	Application Manager UI – Features	Add functions to create, edit, and archive / delete Features.
1.10	Logging	Implement log4net. Add logging to all functionalities built in stages 5-9.

15.2.2 Phase 2 – Environment Manager

The environment manager allows users to configure test environments, control data refreshes, and install application versions.

Stage	Name	Description
2.1	Environment Manager – Configuration	Add functions to create, edit, and archive / delete Environments.
2.2	Environment Manager – Refreshes UI	Implement UI and stub functions / C# interfaces to support refreshes.
2.3	Environment Manager – UI Diagram	Implement UI diagram so users can see the flow between environments for refreshes and will be able to see the version installed in each environment.
2.4	Environment Manager – Refreshes Back-End	Make refreshes work for one in-scope application.
2.5	Environment Manager – Version Installation UI	Allow users to choose versions and install them to environments where installation is permitted.
2.6	Environment Manager – Version Installation Back-End	Make version installation work for one in-scope application.

15.2.3 Phase 3 – Test Configuration User Interface

The Test Configuration User Interface allows users to design and configure test Scenarios.

Stage	Name	Description
3.1	Actions – Structure	Design and implement C# / DB structure for Actions. Support Global, Department, and Application-level Actions. Stubs so properties can be supplied by Scenarios and Workflows.
3.2	Workflows – Structure	Design and implement C# / DB structure for Workflows. Lists of Actions with passing of properties to Actions.
3.3	Workflows and Actions – Implementation	Build several proof-of-concept Workflows and Actions for common, different tasks, EG import file, run processing, compare data. Make a Workflow which can call Actions in sequence.
3.4	Workflow Editor UI	Build UI that allows Workflows to be created and edited.
3.5	Actions – Parallel Processing	Introduce support for parallel processing of Actions within a Workflow.
3.6	Test Cases	Design and implement C# / DB structure for Test Cases.
3.7	Scenarios	Design and implement C# / DB structure for Scenarios.
3.8	Indexing System	Implement the Indexing System for Scenarios and Test Cases. Apply this to the UI so relevant entities display their index.
3.9	Scenarios – Parallel Processing	Introduce support for Workflows within Scenarios to be run in parallel.

Stage	Name	Description
3.10	Properties Passing – Scenarios and Test Cases	Implement ability for Scenarios and Test Cases to pass input values to Actions.
3.11	Properties Passing – Actions	Implement ability for Actions to generate values and pass them as input values to subsequent Actions within the same Workflow.
3.12	Test Case Manager UI	Build the UI that allows users to create Scenarios and Test Cases and configure them.
3.13	SQL Editor Field	Implement reusable UI field that can hold SQL statements. Integrate it into screens for editing Test Cases and Scenarios.
3.14	File / Document Storage System	Implement system that allows artifacts (files and documents) to be stored and managed in SQL Server. This includes the Excel validator.
3.15	Scenario Debugger	Implement UI that allows Scenarios and Workflows to be stepped through and debugged.
3.16	Test Pack Manager UI	Build UI that allows Test Packs to be created, copied, archived, etc.
3.9	Test Results	Design and implement C# / DB structure for Test Results.
33	Data Compare Action and Compare Results Tables	Fully implement Action(s) to perform data compares and write results to Compare Results Tables.
35	Colours Configuration	Implement storage and C# API to manage UI colours (green, orange, red, etc.)

15.2.4 Phase 4 – Results Generator

The Results Generator provides the ability to compare outputs and generate and store results.

Stage	Name	Description
4.1	Test Results	Design and implement C# / DB structure for Test Results.
4.2	Data Compare Action and Compare Results Tables	Fully implement Action(s) to perform data compares and write results to Compare Results Tables.
4.3	Colours Configuration	Implement storage and C# API to manage UI colours (green, orange, red, etc.)

15.2.5 Phase 5 – Test Runner

The Test Runner allows tests to be run, and progress of test runs to be monitored.

Stage	Name	Description
5.1	Test Run Manager UI	Allows Test Runs to be launched. Includes ability to set volume control parameters for Test Runs.
5.2	Real-Time Status View	UI view that updates status of elements within Test Runs in real-time so users can see work completed, work remaining, and results generated so far.
5.3	Test Run Dashboard	Displays history of Test Run results, duration over time. Allows users to determine whether pass rates are improving.

15.2.6 Phase 6 – Results Manager

The Results Manager allows users to review results of a Test Run and drill down into them.

Stage	Name	Description
6.1	Data Compare Viewer	UI that allows users to see the output of Data Compare Actions stored in the Compare Results Tables.
6.2	Tolerances	Allow users to add tolerances in the Data Compare Viewer which clear differences.
6.3	Acceptable Differences	Allow users to add Acceptable Differences in the Data Compare Viewer.
6.4	Break Reports	Allow users to create Break Reports, add comments to them, disseminate them, and manage their workflow.
6.5	Break Report Resolver	Allow users to create Tolerances and Acceptable Differences within the Break Report viewer to close out breaks.
6.6	Break List Viewer	Create a UI allowing users to view, sort, filter, and drill down into Break Reports.
6.7	Test Results Report – Summary, Tolerances, Acceptable Differences	Initial version of the Test Results Report with high-level and sign-off data.
6.8	Test Results Report – Test Results	Core body of the report.
6.9	Test Results Report – Log	Dump of the Test Run log.